COMPUTER ENGINEERING

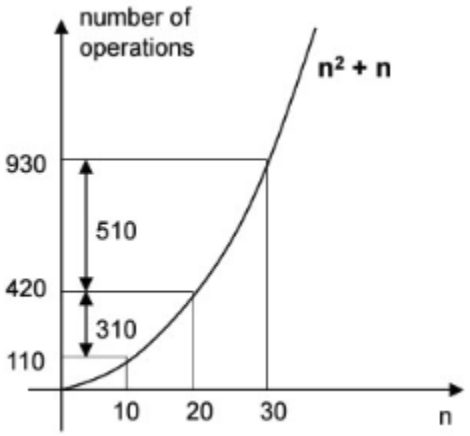
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	07/10/2026
NAME:	Macasaet, John David Milan, Kherwin Emmanuel Moradillo, Eunice	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

ILO A



n (size of array)	number of operations
10	110
20	420
30	930

MACASAET ANALYSIS:

In the worst case scenario where the number of operations is dictated by n^2 + n wherein n is the size of the array. We can see that the difference between each number of operations increases by 200 whenever you increase n by 10.

MILAN ANALYSIS:

The graph of the worst case is n² + n. For n = 10, 20 and 30, the number of operations is 110, 420 and 930. The jumps are 310 and then 510, so the increase is not equal each time.

MORADILLO ANALYSIS:

It shows the worst case which is n^2 + n, where as the values increases by 200 each time when n is also increased by 10.

ILO B			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation**
1000	0 microseconds	`Time taken to search = 0 microseconds` `Student (554 574) needs vaccination.`	With a small input size, the execution speed only takes 0 microseconds to run.
10000	1 microseconds	`Time taken to search = 1 microseconds` `Student (554 574) needs vaccination.`	Idk bro
100000	2 microseconds	`Time taken to search = 2 microseconds` `Student (554 574) needs vaccination.`	As the input size became 100x larger the execution speed slowed by 2 microseconds

Table 11.1

Sa school ginawa namin sir :)

MACASAET ANALYSIS:	**MILAN ANALYSIS:**	**MORADILLO ANALYSIS:**
Based on the table above we can see that the execution speed changes based on the input size and also depends on your CPU. In this particular example, the input size that took the longest time to execute is 100000.	The time to search went up a little as the input size got bigger from 0 to 1 to 2 microseconds but the change is very small even if the list grew. Binary search cuts the list in half every step so the time only grows a little when the size grows.	The output shows that the execution speed decreases as the input size grows larger, from 1000 that only takes 0 microseconds to 2 microseconds that has a 100x larger input size. However, the execution also changes based on the runtime and the program of your computer.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

MACASAET

PROBLEM 1

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis:
Outer Loop: executes n times
Inner Loop: the number of iteration averagely executes about (n-1)/2 times. Getting the average is by adding the first iteration which runs n-1 times and the last term runs 0 times since in the last iteration i equals n-1.
The frequency of operation is denoted by T(N) = n(n-1)/2 by multiplying the number of runs in the outer loop and inner loop.

Experimental Analysis:

```
10 | comparisons=45 | 0 us
20 | comparisons=190 | 1 us
40 | comparisons=780 | 4 us
80 | comparisons=3160 | 14 us
```

In this particular example, the input size doubles each time. And the number of comparisons gets multiplied by approximately 4.

Analysis and Comparison:
We can see in the result for the experimental analysis that it satisfies n(n-1)/2. For example, if the input size is 10 then 10(10-1)/2 which equals to 45.

PROBLEM 2

Theoretical Analysis:

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

rpower
T(N) = n + 1

brpower
T(N) = (n-1)/2 + logn

	Expermental Analysis:
	`rpower n=10 calls=11 100 ns`
	`brpower n=10 calls=5 200 ns`
	`rpower n=20 calls=21 100 ns`
	`brpower n=20 calls=6 200 ns`
	`rpower n=40 calls=41 200 ns`
	`brpower n=40 calls=7 100 ns`
	`rpower n=80 calls=81 300 ns`
	`brpower n=80 calls=8 100 ns`
	For rpower, the number of calls is determined by adding 1 to n. And for brpower, for each call it only gains 1.
	
	Analysis and comparison:
	Both theoretical and experimental roughly gain the same result. For input size 10, 10(10-1)/2 + log10 results in 5.5 which is close to 5.

MILAN	
PROBLEM 1	``` Algorithm Unique(A) for i = 0 to n-1 do for j = i+1 to n-1 do if A[i] equals to A[j] then return false return true ```
	Theoretical Analysis: If A[0] equals A[1]: The comparison executes 1 time, so T(N) = 1, which is O(1) If all elements are unique: When i = 0, the comparison executes n-1 times When i = 1, it executes n-2 times This continues until the last i, which executes 1 time T(N) = (n-1) + (n-2) + ... + 1 = n(n-1)/2, which is $O(n^2)$

	Experimental Analysis:
	`n, time (microseconds)`
	`1000, 1936 (result = true)`
	`2000, 7774 (result = true)`
	`4000, 32051 (result = true)`
	`8000, 129168 (result = true)`
	`16000, 506445 (result = true)`
	I used std::chrono (steady_clock) to time the function. The array was all unique values so it is the worst case. The array sizes were doubled each time.

	Analysis and Comparison: the comparisons grow with n^2, so when n doubles, the work should be about 4 times. In the experiment, the time went up about 4 times each time n doubled

PROBLEM 2	`Algorithm rpower(int x, int n):`
	`1 if n == 0 return 1`
	`2 else return x*rpower(x, n-1)`
	
	`Algorithm brpower(int x, int n):`
	`1 if n == 0 return 1`
	`2 if n is odd then`
	`3  y = brpower(x, (n-1)/2)`
	`4  return x*y*y`
	`5 if n is even then`
	`6  y = brpower(x, n/2)`
	`7  return y*y`

	Theoretical Analysis: If n is 0 in rpower: Line 1 executes 1 time Return 1 executes 1 time. T(0) = 1 + 1 = 2 If n is more than 0 in rpower Line 1 executes 1 time Line 2 executes 1 time Recursive rpower(x, n-1) executes T(n-1) T(n) = T(n-1) + 2 brpower:

	If n is 0: Line 1 executes 1 time Return 1 executes 1 time T(0) = 1 + 1 = 2 If n is odd: Line 1 and Line 2 (comparisons) execute 1 time each, which is 2 Line 3 (call and assignment) executes 1 time plus the recursive brpower(x, (n-1)/2) Line 4 (return) executes 1 time T(n) = T((n-1)/2) + 4 If n is even: Line 1, Line 2 and Line 5 execute 1 time each, which is 3 Line 6 executes 1 time plus the recursive brpower(x, n/2) Line 7 (return) executes 1 time T(n) = T(n/2) + 5
	Experimental Analysis: n, rpower calls, rpower time (ns), brpower calls, brpower time (ns) 10, 11, 200, 5, 100 (same answer: yes) 100, 101, 600, 8, 100 (same answer: yes) 1000, 1001, 25700, 11, 100 (same answer: yes) 5000, 5001, 130100, 14, 200 (same answer: yes) 10000, 10001, 166400, 15, 200 (same answer: yes) I used std::chrono to time rpower and brpower with the same x and n, and I also counted the number of recursive calls.
	Analysis and Comparison: rpower made n+1 calls every time (11, 101, 1,001, 5,001 and 10,001). This matches the theory that it grows with n, T(N) = 2n + 2. brpower made only 5, 8, 11, 14 and 15 calls.

MORADILLO	
PROBLEM 1	``` Algorithm Unique(A) for i = 0 to n-1 do for j = i+1 to n-1 do if A[i] equals to A[j] then return false return true ```

Theoretical Analysis:
If n is equal to 0
T(0) = 1 + 1
T(N) = 2

If n equals 0 then (n-1)
If n equals 1 then (n-2)
Until the last i
T(N) = (n+1) + n(n+1)/2 = n(n-1)/2 = 0 +1
T(N) = n ^2 + n = 2 = O(n^2)

Experimental Analysis:

```
n       comparisons     n(n-1)/2        time (us)
1000    499500          499500          1964
2000    1999000         1999000         8088
4000    7998000         7998000         32501
8000    31996000                31996000                128401
16000   127992000               127992000               512560

Best case (A[0] == A[1])
n       comparisons     time (us)
1000    1               0
2000    1               0
4000    1               0
8000    1               0
16000   1               0
```

Using various inputs, each has a unique value of arrays that doubles each time.

Analysis and Comparison:
It is seen from the experimental analysis that it uses the n(n-1) / 2 that matches the results of the theoretical analysis that predicts the worst case.

PROBLEM 2

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis:

Base Case
T(0) = 1

If n is equal to 0 in rpower

	T(N) = 1+1 T(0)= 2 If n is odd Line 1,2 ,4 run T(N) = T((n-1)/2) +3 If n is even Line 1,2,5,7 run T(N) = T(N/2) + 4

	Experimental Analysis:
	`N = 100`
	`Time taken to execute rpower = 400 nanoseconds`
	`Time taken to execute brpower = 200 nanoseconds`
	`N = 1000`
	`Time taken to execute rpower = 26600 nanoseconds`
	`Time taken to execute brpower = 200 nanoseconds`
	`N = 10000`
	`Time taken to execute rpower = 255300 nanoseconds`
	`Time taken to execute brpower = 200 nanoseconds`
	It shows the operations of rpower and bpower

	Analysis and Comparison: This confirms the both the theory that rpower is o(n) because its time doubled then doubled and bwpoer is o(log n) because it says th same

	CONCLUSION
MACASAET	In this activity, we learned about the runtime analysis and how it is computed. There is both theoretical and experimental analysis, the former involves time complexity and the latter needs coding for you to get the results.
MILAN	In this activity, I learned that we can check how fast an algorithm is by counting operations and by timing it with chrono. Both ways gave the same result. Binary search and brpower are O(log n) and stay fast, rpower is O(n), and Unique and diff are $O(n^2)$ and get slow when n is big.
MORADILLO	In this activity, I learned that an algorithm's running time can be analyzed in theory by counting operations and checked in practice by timing it with std::chrono, and that both ways showed the same growth: binary search is O(log n), Unique is $O(n^2)$, rpower is O(n), and brpower is O(log n). The experiments matched the theory, though the exact times varied between runs because they depend

	on the computer, so counting operations was the more reliable measure. I think I did well on the analysis, and I want to improve at solving recurrences and at writing cleaner test code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.